PYTHON FOR BEGINNERS · SESSION 14

File Handling

Read, write, append, with, and working with text and CSV files.

Recap

Ask the class:

- What is the difference between `import math` and `from math import sqrt`?
- What does `random.randint(1, 6)` give?
- Where do you type `pip install`?

Why files?

Variables live in **memory**. When the program ends, everything is **gone**. A **file** keeps data on the disk, so it is still there next time.

- Save marks, contacts, a to-do list or a game score.
- Read data that someone else made, such as a list of names.
- Write a **report** or a **log** of what the program did.

> **Files in the Try it boxes**
>
> The Try it boxes run inside your browser and use a **temporary** file area. Files you write there are real for that page, but they disappear when you reload. On your own computer the files stay on the disk.

open() and the modes

`open(name, mode)` opens a file and gives back a **file object**. The **mode** says what you want to do.

MODE	MEANING	IF THE FILE DOES NOT EXIST
`"r"`	read (the default)	error
`"w"`	write: **erases** the old content	creates it
`"a"`	append: adds to the **end**	creates it
`"x"`	create new, fail if it exists	creates it
`"r+"`	read and write	error

Add `"b"` for a **binary** file such as an image (`"rb"`, `"wb"`). Without it the file is **text**.

Writing a file

```
f = open("hello.txt", "w")
f.write("Hello, file!")
f.close()

f = open("hello.txt", "r")
print(f.read())
f.close()
```

```
OUTPUT
Hello, file!
```

- `"w"` makes the file if it is not there.
- `write()` puts text in the file. It does **not** add a new line for you.
- `close()` finishes the job. Always close a file you open.

New lines

Put `\n` at the end of each line to start the next line.

```
f = open("names.txt", "w")
f.write("Aman\n")
f.write("Ankit\n")
f.write("Neha\n")
f.close()

print(open("names.txt").read())
```

```
OUTPUT
Aman
Ankit
Neha
```

writelines

```
lines = ["one\n", "two\n", "three\n"]

f = open("numbers.txt", "w")
f.writelines(lines)
f.close()

print(open("numbers.txt").read())
```

```
OUTPUT
one
two
three
```

`writelines` writes a list of strings, but it does **not** add the `\n` for you. Put it in each string.

The with statement

If you forget `close()`, or the program crashes in the middle, the file can stay open. The `with` statement closes it **automatically** when the block ends. This is the **best way** to work with files.

```
with open("notes.txt", "w") as f:
    f.write("Line 1\n")
    f.write("Line 2\n")

with open("notes.txt") as f:
    print(f.read())
```

```
OUTPUT
Line 1
Line 2
```

- `as f` gives the file object the name `f`.
- The indented lines run while the file is open.
- When the indent ends, Python closes the file for you.

```
with open("data.txt", "w") as f:
    f.write("hi")

print(f.closed)
```

```
OUTPUT
True
```

`f.closed` is `True` after the `with` block. The file is shut for you.

Reading a file

First, make a small file to practise on. The next boxes each build their own copy.

METHOD	WHAT IT GIVES
`f.read()`	the **whole file** as one string
`f.readline()`	the **next line** (with its `\n`)
`f.readlines()`	a **list** of all the lines
`for line in f:`	one line at a time (best for big files)

```
with open("fruits.txt", "w") as f:
    f.write("apple\nbanana\ncherry\n")

with open("fruits.txt") as f:
    text = f.read()
print(text)
print(len(text))
```

```
OUTPUT
apple
banana
cherry

20
```

readline and readlines

```
with open("fruits.txt", "w") as f:
    f.write("apple\nbanana\ncherry\n")

with open("fruits.txt") as f:
    print(f.readline())
    print(f.readline())

with open("fruits.txt") as f:
    print(f.readlines())
```

```
OUTPUT
apple

banana

['apple\n', 'banana\n', 'cherry\n']
```

Each line ends with `\n`. That is why `print` leaves a blank line after the first two lines.

Looping over the lines

```
with open("fruits.txt", "w") as f:
    f.write("apple\nbanana\ncherry\n")

with open("fruits.txt") as f:
    for line in f:
        print(line.strip())
```

OUTPUT

```
apple
banana
cherry
```

`strip()` removes the `\n` (and spaces) at the ends of each line.

Reading numbers

Everything you read from a text file is a **string**. Convert with `int()` or `float()` before doing maths.

```
with open("marks.txt", "w") as f:
    f.write("70\n85\n90\n")

total = 0
with open("marks.txt") as f:
    for line in f:
        total += int(line)

print("Total:", total)
```

OUTPUT

```
Total: 245
```

The file position

A file remembers **where you are** in it. After `read()` you are at the **end**, so a second `read()` gives an **empty string**.

Work out the output first, then press Run to check

```
with open("p.txt", "w") as f:
    f.write("abc")

with open("p.txt") as f:
    print(repr(f.read()))
    print(repr(f.read()))
    f.seek(0)
    print(repr(f.read()))
```

- `f.seek(0)` jumps back to the start.
- `f.tell()` tells you the current position.
- To read the file again, just open it again in a new `with`.

Appending

`"w"` **wipes** the old content. To **add** to the end, use `"a"`.

```
with open("log.txt", "w") as f:
    f.write("Start\n")

with open("log.txt", "a") as f:
    f.write("Step 1\n")
    f.write("Step 2\n")

with open("log.txt") as f:
    print(f.read())
```

OUTPUT

```
Start
Step 1
Step 2
```

Work out the output first, then press Run to check

```
with open("w.txt", "w") as f:
    f.write("first\n")

with open("w.txt", "w") as f:
    f.write("second\n")

with open("w.txt") as f:
    print(f.read())
```

"w" erases the file

Opening an existing file with `"w"` throws away what was in it, **even if you never write anything**. Use `"a"` when you want to keep the old content.

Examples with text files

Count lines and words

```
with open("poem.txt", "w") as f:
    f.write("Roses are red\nViolets are blue\nPython is fun\n")

lines = 0
words = 0
with open("poem.txt") as f:
    for line in f:
        lines += 1
        words += len(line.split())

print("Lines:", lines)
print("Words:", words)
```

OUTPUT

```
Lines: 3
Words: 9
```

Search for a word

```
with open("poem.txt", "w") as f:
    f.write("Roses are red\nViolets are blue\nPython is fun\n")

with open("poem.txt") as f:
    for number, line in enumerate(f, start=1):
        if "are" in line:
            print(number, line.strip())
```

OUTPUT

```
1 Roses are red
2 Violets are blue
```

Copy a file with changes

```
with open("small.txt", "w") as f:
    f.write("hello\nworld\n")

with open("small.txt") as src, open("big.txt", "w") as dst:
    for line in src:
        dst.write(line.upper())

with open("big.txt") as f:
    print(f.read())
```

```
OUTPUT
HELLO
WORLD
```

One `with` can open **two files** at once, separated by a comma.

Does the file exist?

Opening a file that is not there for reading gives a `FileNotFoundError`.

Try it: this fails on purpose

```
with open("missing.txt") as f:
    print(f.read())
```

```
ERROR
  File "your code", line 1, in <module>
FileNotFoundError: [Errno 44] No such file or directory: 'missing.txt'
```

You can check first with the `os` module.

```python
import os

print(os.path.exists("missing.txt"))

with open("here.txt", "w") as f:
    f.write("hi")

print(os.path.exists("here.txt"))
print(os.path.getsize("here.txt"))

os.remove("here.txt")
print(os.path.exists("here.txt"))
```

OUTPUT

```
False
True
2
False
```

NAME	WHAT IT DOES
`os.path.exists(p)`	`True` if the file or folder is there
`os.path.getsize(p)`	size in bytes
`os.remove(p)`	delete a file
`os.rename(a, b)`	rename a file
`os.listdir()`	names of the files in the folder

Next class

There is a better way to handle a missing file than checking first: `try` and `except`. We learn it in the next session.

Paths and encoding

A **path** says where a file is. A name like `"data.txt"` means "in the **current folder**" (the folder the program runs from).

- **Relative path:** `"data/marks.txt"` starts from the current folder.
- **Absolute path:** `"C:/Users/Aman/marks.txt"` starts from the drive.

Backslashes on Windows

Write `"C:/Users/Aman/marks.txt"` with **forward slashes**, or a raw string `r"C:\Users\Aman\marks.txt"`. A plain `"C:\new\table.txt"` breaks, because `\n` and `\t` are special characters.

pathlib

The `pathlib` module treats a path as an object. `write_text` and `read_text` do a whole file in one line.

```
from pathlib import Path

p = Path("quick.txt")
p.write_text("Short and sweet")
print(p.read_text())
print(p.exists())
print(p.suffix)
```

```
OUTPUT
Short and sweet
True
.txt
```

Encoding

A text file stores letters as numbers using an **encoding**. Use `encoding="utf-8"` so names like `Zoë` or Hindi text are safe on every computer.

```
with open("name.txt", "w", encoding="utf-8") as f:
    f.write("Zoë")

with open("name.txt", encoding="utf-8") as f:
    print(f.read())
```

```
OUTPUT
Zoë
```

Example: a to-do list

A to-do list that **keeps its tasks** between runs. It uses `input()`, so run it on your own computer.

todo.py (run it on your computer)

```
FILE = "todo.txt"

def load():
    tasks = []
    try:
        with open(FILE) as f:
            for line in f:
                tasks.append(line.strip())
    except FileNotFoundError:
        pass
    return tasks

def save(tasks):
    with open(FILE, "w") as f:
        for task in tasks:
            f.write(task + "\n")

tasks = load()
print("You have", len(tasks), "tasks")

new = input("New task (or press Enter to skip): ")
if new:
    tasks.append(new)
    save(tasks)

for i, task in enumerate(tasks, start=1):
    print(i, task)
```

OUTPUT

```
You have 0 tasks
New task (or press Enter to skip): Buy milk
1 Buy milk
```

Run it twice. The second run starts with `You have 1 tasks`, because the list was **saved in a file**. (The `try` and `except` part is next class.)

The same idea, without input()

```
def save(tasks):
    with open("todo.txt", "w") as f:
        for task in tasks:
            f.write(task + "\n")

def load():
    with open("todo.txt") as f:
        return [line.strip() for line in f]

save(["Buy milk", "Call Neha", "Study Python"])
tasks = load()
tasks.append("Do homework")
save(tasks)

for i, task in enumerate(load(), start=1):
    print(i, task)
```

OUTPUT

```
1 Buy milk
2 Call Neha
3 Study Python
4 Do homework
```

CSV files

CSV means **comma separated values**. It is a plain text file for a **table**: one row per line, columns separated by commas. Excel and Google Sheets open it.

marks.csv

```
name,maths,science
Aman,85,90
Ankit,70,65
Neha,92,88
```

You could split each line yourself, but the `csv` module handles quotes and commas inside values for you.

Writing a CSV

```
import csv

rows = [
    ["name", "maths", "science"],
    ["Aman", 85, 90],
    ["Ankit", 70, 65],
    ["Neha", 92, 88],
]

with open("marks.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(rows)

print(open("marks.csv").read())
```

OUTPUT

```
name,maths,science
Aman,85,90
Ankit,70,65
Neha,92,88
```

- `newline=""` stops blank lines appearing between rows on Windows. Always use it with `csv`.
- `writerow(list)` writes one row. `writerows(list of lists)` writes many.

Reading a CSV

```
import csv

with open("marks.csv", "w", newline="") as f:
    csv.writer(f).writerows([["name", "maths"], ["Aman", 85], ["Ankit", 70]])

with open("marks.csv", newline="") as f:
    for row in csv.reader(f):
        print(row)
```

OUTPUT

```
['name', 'maths']
['Aman', '85']
['Ankit', '70']
```

Each row is a **list of strings**. The numbers come back as text, such as `"85"`. Convert them with `int()`.

Skipping the header

```
import csv

with open("marks.csv", "w", newline="") as f:
    csv.writer(f).writerows([["name", "maths"], ["Aman", 85], ["Ankit", 70]])

with open("marks.csv", newline="") as f:
    reader = csv.reader(f)
    header = next(reader)
    print("Columns:", header)
    for name, maths in reader:
        print(name, int(maths) + 5)
```

```
OUTPUT
Columns: ['name', 'maths']
Aman 90
Ankit 75
```

`next(reader)` takes the first row (the header) off, so the loop starts at the data.

DictReader and DictWriter

Instead of numbering the columns, use **names**. `DictReader` gives each row as a **dictionary**, using the header as the keys.

```
import csv

with open("marks.csv", "w", newline="") as f:
    csv.writer(f).writerows([
        ["name", "maths", "science"],
        ["Aman", 85, 90],
        ["Ankit", 70, 65],
        ["Neha", 92, 88],
    ])

with open("marks.csv", newline="") as f:
    for row in csv.DictReader(f):
        print(row["name"], row["maths"])
```

```
OUTPUT
Aman 85
Ankit 70
Neha 92
```

DictWriter

```
import csv

students = [
    {"name": "Aman", "city": "Delhi"},
    {"name": "Neha", "city": "Pune"},
]

with open("people.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "city"])
    writer.writeheader()
    writer.writerows(students)

print(open("people.csv").read())
```

OUTPUT

```
name,city
Aman,Delhi
Neha,Pune
```

`fieldnames` sets the column order. `writeheader()` writes the first line.

Example: marks report

Read a CSV of marks, work out each student's total and average, find the topper, and write a **report file**.

```
import csv

with open("marks.csv", "w", newline="") as f:
    csv.writer(f).writerows([
        ["name", "maths", "science", "english"],
        ["Aman", 85, 90, 78],
        ["Ankit", 70, 65, 80],
        ["Neha", 92, 88, 95],
    ])

results = []
with open("marks.csv", newline="") as f:
    for row in csv.DictReader(f):
        marks = [int(row["maths"]), int(row["science"]), int(row["english"])]
        results.append((row["name"], sum(marks), sum(marks) / 3))

topper = max(results, key=lambda r: r[1])

with open("report.txt", "w") as out:
    for name, total, avg in results:
        out.write(f"{name}: total {total}, average {avg:.1f}\n")
    out.write(f"Topper: {topper[0]}\n")

print(open("report.txt").read())
```

OUTPUT

```
Aman: total 253, average 84.3
Ankit: total 215, average 71.7
Neha: total 275, average 91.7
Topper: Neha
```

- `max(..., key=lambda ...)` from the advanced functions class finds the top total.
- We read one file and wrote another in the same program.

Common mistakes

- **Opening with `"w"` when you meant `"a"`.** The old content is gone.
- **Forgetting `\n`.** All the text ends up on **one line**.
- **Forgetting `close()`.** Use `with` and you never forget.
- **Reading twice.** The second `read()` is empty. Open the file again, or use `seek(0)`.
- **Using numbers from a file as numbers.** They are strings. Use `int()` first.
- **Writing a number.** `f.write(5)` is a `TypeError`. Write `str(5)`.
- **Missing `newline=""` with csv** (blank lines on Windows).
- **Wrong folder.** The file name is relative to where the program **runs**, not where the `.py` file sits.

Try it: this fails on purpose

```
with open("number.txt", "w") as f:
    f.write(5)
```

```
ERROR
  File "your code", line 2, in <module>
TypeError: write() argument must be str, not int
```

Practice

Your turn: write the numbers 1 to 5, one per line, into nums.txt, then print the file

```
with open("nums.txt", "w") as f:
    pass
```

Your turn: read nums.txt and print the sum of the numbers

```
with open("nums.txt", "w") as f:
    f.write("1\n2\n3\n4\n5\n")
```

Your turn: append the line "6" to nums.txt, then print the number of lines

```
with open("nums.txt", "w") as f:
    f.write("1\n2\n3\n4\n5\n")
```

Homework and wrap-up

- Write a program that saves **5 names** typed by the user into `names.txt`, one per line.
- Read a text file and print the **longest line** and the **number of words**.
- Make a **contact book** that saves `name,phone` rows in a CSV and can search by name.
- Read `marks.csv` and write a second CSV with the **total and average** added as new columns.
- Make a **diary** program: each run **appends** today's entry with the date from `datetime`.
- Revise: **modes** `r`, `w`, `a`, `with`, `read`, `readline`, `readlines`, `csv.reader`, `DictReader`.

Next class

Exception handling: `try`, `except`, `finally`, common errors and debugging tips.